Recursion

1. What Is Recursion?

2. The Three Laws of Recursion

3. Tower of Hanoi

4. Exploring the maze

5.2 What Is Recursion?

Recursion is a method of solving problems that involves **breaking a problem down into smaller and smaller subproblems** until you get to a small enough problem that it can be solved trivially.

Recursion is a method of solving problems that involves **breaking a problem down into smaller and smaller subproblems** until you get to a small enough problem that it can be solved trivially.

Usually recursion involves a function calling itself. It allows us to write elegant solutions to problems that may otherwise be very difficult to program!

5.3 Calculating the Sum of a List of Numbers

Suppose that you want to calculate the sum of a vector of numbers such as: `{1, 3, 5, 7, 9}`. An iterative function that computes the sum is shown below. The function uses an accumulator variable (`theSum`) to compute a running total by starting with 0 and adding each number in the vector.

Suppose that you want to calculate the sum of a vector of numbers such as: `{1, 3, 5, 7, 9}`. An iterative function that computes the sum is shown below. The function uses an accumulator variable (`theSum`) to compute a running total by starting with 0 and adding each number in the vector.

In [1]:
```
#include <iostream>
#include <vector>
using namespace std;

int listSum(vector<int> numList) {
    int theSum = 0;
    for (int i : numList) {
        theSum = theSum + i;
    }
    return theSum;
}

int main() {
    cout << listSum({1, 3, 5, 7, 9}) << endl;
    return 0;
}
```

25

Suppose that you want to calculate the sum of a vector of numbers such as: `{1, 3, 5, 7, 9}`. An iterative function that computes the sum is shown below. The function uses an accumulator variable (`theSum`) to compute a running total by starting with 0 and adding each number in the vector.

In [1]:
```cpp
#include <iostream>
#include <vector>
using namespace std;

int listSum(vector<int> numList) {
    int theSum = 0;
    for (int i : numList) {
        theSum = theSum + i;
    }
    return theSum;
}

int main() {
    cout << listSum({1, 3, 5, 7, 9}) << endl;
    return 0;
}
```

25

The function uses an accumulator variable (`the_sum`) to compute a running total of all the numbers in the list by starting with 0.

If we do not have loops. How would you compute the sum of a list of numbers?

If we do not have loops. How would you compute the sum of a list of numbers?

You might start by recalling that addition is a function that is defined for two parameters, a pair of numbers. To redefine the problem from adding a list to adding pairs of numbers, we could rewrite the list as a **fully parenthesized expression**. Such an expression looks like this:

If we do not have loops. How would you compute the sum of a list of numbers?

You might start by recalling that addition is a function that is defined for two parameters, a pair of numbers. To redefine the problem from adding a list to adding pairs of numbers, we could rewrite the list as a **fully parenthesized expression**. Such an expression looks like this:

$$(1 + (3 + (5 + (7 + 9))))$$

Notice that the innermost set of parentheses, , is a problem that we can solve without a loop or any special constructs! In fact, we can use the following sequence of simplifications to compute a final sum:

Notice that the innermost set of parentheses, $(7 + 9)$, is a problem that we can solve without a loop or any special constructs! In fact, we can use the following sequence of simplifications to compute a final sum:

$$\begin{aligned} total = &\ (1 + (3 + (5 + (7 + 9)))) \\ total = &\ (1 + (3 + (5 + 16))) \\ total = &\ (1 + (3 + 21)) \\ total = &\ (1 + 24) \\ total = &\ 25 \end{aligned}$$

First, let's restate the sum problem in terms of `C++ vectors`. We might say the sum of the vector `numList` is the sum of the first element of the vector (`numList[0]`) and the sum of the rest of the vector. The rest of the vector can be built with the iterator-range constructor.

First, let's restate the sum problem in terms of `C++ vectors`. We might say the sum of the vector `numList` is the sum of the first element of the vector (`numList[0]`) and the sum of the rest of the vector. The rest of the vector can be built with the iterator-range constructor.

$$list\_sum(num\_list) = first(num\_list) + list\_sum(rest(num\_list))$$

First, let's restate the sum problem in terms of `C++` `vectors`. We might say the sum of the vector `numList` is the sum of the first element of the vector (`numList[0]`) and the sum of the rest of the vector. The rest of the vector can be built with the iterator-range constructor.

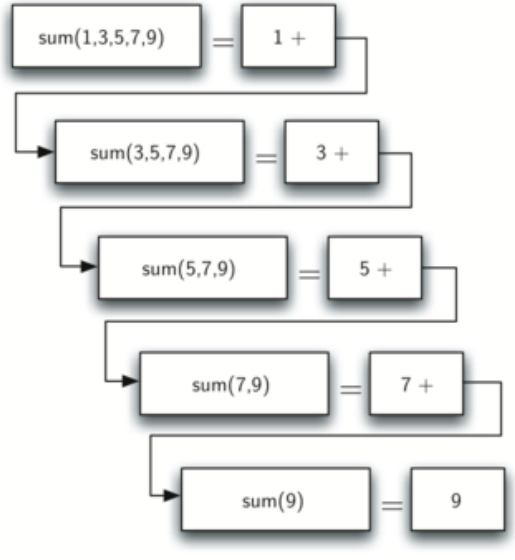
$$list\_sum(num\_list) = first(num\_list) + list\_sum(rest(num\_list))$$

In [2]:
```cpp
#include <iostream>
#include <vector>
using namespace std;

int listSum(vector<int> numList) {
    if (numList.size() == 1) {
        return numList[0];
    } else {
        vector<int> rest(numList.begin() + 1, numList.end());   // numList[1:]
        return numList[0] + listSum(rest);
    }
}

int main() {
    cout << listSum({1, 3, 5, 7, 9}) << endl;
    return 0;
}
```

25

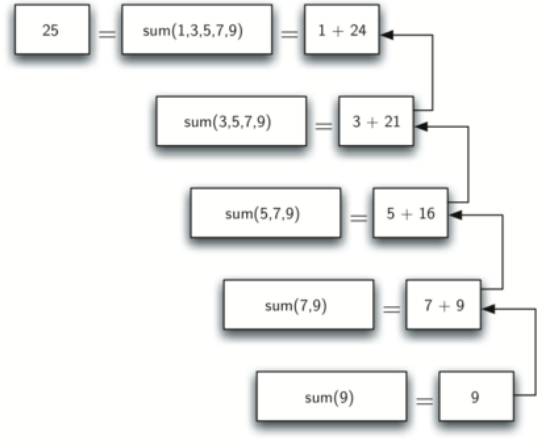
Figure below shows the series of recursive calls that are needed to sum the list. You should think of this series of calls as a series of simplifications.

Figure below shows the series of recursive calls that are needed to sum the list. You should think of this series of calls as a series of simplifications.



When we reach the point where the problem is as simple as it can get, we begin to piece together the solutions of each of the small problems until the initial problem is solved!

When we reach the point where the problem is as simple as it can get, we begin to piece together the solutions of each of the small problems until the initial problem is solved!



5.4 The Three Laws of Recursion

Like robots in Asimov's stories, all recursive algorithms must obey three important laws:

1. A recursive algorithm must have a base case.

Like robots in Asimov's stories, all recursive algorithms must obey three important laws:

1. A recursive algorithm must have a base case.

2. A recursive algorithm must change its state and move toward the base case.

Like robots in Asimov's stories, all recursive algorithms must obey three important laws:

1. A recursive algorithm must have a base case.

2. A recursive algorithm must change its state and move toward the base case.

3. A recursive algorithm must call itself recursively.

First, a base case is the condition that allows the algorithm to stop recursing. A base case is typically a problem that is small enough to solve directly. In the `list_sum()` algorithm the base case is a list of length 1.

First, a base case is the condition that allows the algorithm to stop recursing. A base case is typically a problem that is small enough to solve directly. In the `list_sum()` algorithm the base case is a list of length 1.

A change of state means that some data that the algorithm is using is modified. Usually the data that represents our problem gets smaller in some way. In the `list_sum()` algorithm our primary data structure is a list. Since the base case is a list of length 1, a natural progression toward the base case is to shorten the list.

First, a base case is the condition that allows the algorithm to stop recursing. A base case is typically a problem that is small enough to solve directly. In the `list_sum()` algorithm the base case is a list of length 1.

A change of state means that some data that the algorithm is using is modified. Usually the data that represents our problem gets smaller in some way. In the `list_sum()` algorithm our primary data structure is a list. Since the base case is a list of length 1, a natural progression toward the base case is to shorten the list.

The final law is that the algorithm must call itself. This is the very definition of recursion.

As a novice programmer, you have learned that functions are good because you can take a large problem and break it up into smaller problems. The smaller problems can be solved by writing a function to solve each problem.

As a novice programmer, you have learned that functions are good because you can take a large problem and break it up into smaller problems. The smaller problems can be solved by writing a function to solve each problem.

When we talk about recursion it may seem that we are talking ourselves in circles. We have a problem to solve with a function, but that function solves the problem by calling itself! We are solving a problem by breaking it down into a smaller and easier problems.

In [ ]:
```
display_quiz(path+"recursive1.json", max_width=800)
```

How many recursive calls are made when computing the sum of the vector {2, 4, 6, 8, 10}?

- ○ 5
- ○ 3
- ○ 6
- ○ 4

In [ ]:
```
display_quiz(path+"recursive2.json", max_width=800)
```

Suppose you are going to write a recursive function to calculate the factorial of a number. fact(n) returns n * (n-1) * (n-2) * ... where the factorial of zero is defined to be 1. What would be the most efficient and appropriate base case?

- ○ `n == 1`
- ○ `n >= 0`
- ○ `n == 0`
- ○ `n <= 1`

5.5 Converting an Integer to a String in Any Base

Suppose you want to convert an integer to a string in some base between binary and hexadecimal. For example, convert the integer 10 to its string representation in decimal as "10", or to its string representation in binary as "1010".

Suppose you want to convert an integer to a string in some base between binary and hexadecimal. For example, convert the integer 10 to its string representation in decimal as "10", or to its string representation in binary as "1010".

While there are many algorithms to solve this problem, including the algorithm discussed in the stack section, the recursive formulation of the problem is very elegant.

Suppose you want to convert an integer to a string in some base between binary and hexadecimal. For example, convert the integer 10 to its string representation in decimal as "10", or to its string representation in binary as "1010".

While there are many algorithms to solve this problem, including the algorithm discussed in the stack section, the recursive formulation of the problem is very elegant.

Let's look at a concrete example using base `10` and the number `769`. Suppose we have a sequence of characters corresponding to the first 10 digits, like `convert_string = "0123456789"`. It is easy to convert a number less than 10 to its string equivalent by looking it up in the sequence.

If we can arrange to break up the number `769` into three single-digit numbers, `7`, `6`, and `9`, then converting it to a string is simple. A number less than 10 sounds like a good base case.

If we can arrange to break up the number `769` into three single-digit numbers, `7`, `6`, and `9`, then converting it to a string is simple. A number less than 10 sounds like a good base case.

Knowing what our base is suggests that the overall algorithm will involve three components:

If we can arrange to break up the number `769` into three single-digit numbers, `7`, `6`, and `9`, then converting it to a string is simple. A number less than 10 sounds like a good base case.

Knowing what our base is suggests that the overall algorithm will involve three components:

1. Reduce the original number to a series of single-digit numbers.
2. Convert the single digit-number to a string using a lookup.
3. Concatenate the single-digit strings together to form the final result.

The next step is to figure out how to change state and make progress toward the base case.

The next step is to figure out how to change state and make progress toward the base case.

Let's consider what mathematical operations might reduce a number. The most likely candidates are division and subtraction. While subtraction might work, it is unclear what we should subtract from what. Integer division with remainders gives us a clear direction!

The next step is to figure out how to change state and make progress toward the base case.

Let's consider what mathematical operations might reduce a number. The most likely candidates are division and subtraction. While subtraction might work, it is unclear what we should subtract from what. Integer division with remainders gives us a clear direction!

Using integer division to divide `769` by `10`, we get `76` with a remainder of `9`. This gives us two good results.

1. The remainder is a number less than our base that can be converted to a string immediately by lookup.

The next step is to figure out how to change state and make progress toward the base case.

Let's consider what mathematical operations might reduce a number. The most likely candidates are division and subtraction. While subtraction might work, it is unclear what we should subtract from what. Integer division with remainders gives us a clear direction!

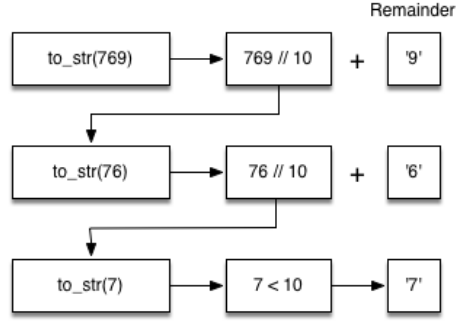
Using integer division to divide `769` by `10`, we get `76` with a remainder of `9`. This gives us two good results.

1. The remainder is a number less than our base that can be converted to a string immediately by lookup.

2. We get a number that is smaller than our original and moves us toward the base case of having a single number less than our base.

Now our job is to convert `76` to its string representation. Again we will use integer division plus remainder to get results of `7` and `6` respectively.

Now our job is to convert `76` to its string representation. Again we will use integer division plus remainder to get results of `7` and `6` respectively.



In [3]:

```cpp
#include <iostream>
#include <string>
using namespace std;

// The algorithm outlined above for any base between 2 and 16.
string toStr(int n, int base) {
    string convertString = "0123456789ABCDEF";
    if (n < base) {
        return string(1, convertString[n]);
    } else {
        return toStr(n / base, base) + convertString[n % base];
    }
}

int main() {
    cout << toStr(1453, 16) << endl;
    return 0;
}
```

5AD

In [3]:
```cpp
#include <iostream>
#include <string>
using namespace std;

// The algorithm outlined above for any base between 2 and 16.
string toStr(int n, int base) {
    string convertString = "0123456789ABCDEF";
    if (n < base) {
        return string(1, convertString[n]);
    } else {
        return toStr(n / base, base) + convertString[n % base];
    }
}

int main() {
    cout << toStr(1453, 16) << endl;
    return 0;
}
```

5AD

Notice that in line 3 we check for the base case where `n` is less than the base we are converting to. When we detect the base case, we stop recursing and simply return the string from the `convert_string` sequence.

In [3]:
```cpp
#include <iostream>
#include <string>
using namespace std;

// The algorithm outlined above for any base between 2 and 16.
string toStr(int n, int base) {
    string convertString = "0123456789ABCDEF";
    if (n < base) {
        return string(1, convertString[n]);
    } else {
        return toStr(n / base, base) + convertString[n % base];
    }
}

int main() {
    cout << toStr(1453, 16) << endl;
    return 0;
}
```

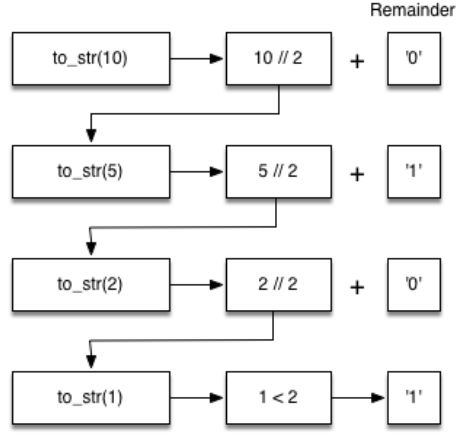
5AD

Notice that in line 3 we check for the base case where `n` is less than the base we are converting to. When we detect the base case, we stop recursing and simply return the string from the `convert_string` sequence.

In line 6 we satisfy both the second and third laws – by making the recursive call and by reducing the problem size–using division.

Let's trace the algorithm; this time we will convert the number `10` to its base 2 string representation (`"1010"`):

Let's trace the algorithm; this time we will convert the number `10` to its base 2 string representation (`"1010"`):



It looks like the digits are in the wrong order. The algorithm works correctly because we make the recursive call first on line 6, then we add the string representation of the remainder.

It looks like the digits are in the wrong order. The algorithm works correctly because we make the recursive call first on line 6, then we add the string representation of the remainder.

If we reversed returning the `convert_string` lookup and returning the `to_str()` call, the resulting string would be backward! But by delaying the concatenation operation until after the recursive call has returned, we get the result in the proper order! This should remind you of our discussion of stacks back in the previous chapter.

Exercsie: Write a function using recursion that takes a string as a parameter and returns a new string that is the reverse of the old string.

In [4]:

```cpp
#include <iostream>
#include <cassert>
using namespace std;

string reverse(string s) {
    if (s.length() == 1) {
        return ____;
    }
    return ____ + s[0];
}

int main() {
    assert(reverse("hello") == "olleh");
    cout << "Pass" << endl;
    return 0;
}
```

```
/tmp/tmpgw0970nz.cpp: In function ‘std::string reverse(std::string)’:
/tmp/tmpgw0970nz.cpp:9:16: error: ‘____’ was not declared in this scop
e
    9 |         return ____;
      |                ^~~~

/tmp/tmpgw0970nz.cpp:11:12: error: ‘____’ was not declared in this sco
pe
   11 |     return ____ + s[0];
      |            ^~~~
```

```
[C++ kernel] Error: Unable to compile the source code. Return error: 0
x1.
```

5.6 Stack Frames: Implementing Recursion

Suppose that instead of concatenating the result of the recursive call to `to_str()` with the string from `convert_string`, we modified our algorithm to push the strings onto a stack instead of making the recursive call:

Using the STL stack, we can replace recursion with an explicit stack of characters:

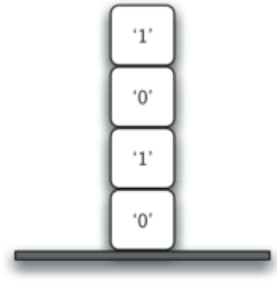
Using the STL stack, we can replace recursion with an explicit stack of characters:

In [5]:
```cpp
#include <iostream>
#include <stack>
#include <string>
using namespace std;

string toStr(int n, int base) {
    stack<char> rStack;
    string convertString = "0123456789ABCDEF";
    while (n > 0) {
        rStack.push(convertString[n % base]);
        n = n / base;
    }
    string res = "";
    while (!rStack.empty()) {
        res = res + rStack.top();
        rStack.pop();
    }
    return res;
}

int main() { cout << toStr(1453, 16) << endl; }
```

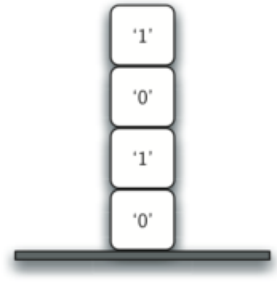
5AD

Each time we make a call to `to_str()`, we push a character on the stack. Returning to the previous example we can see that after the fourth call to `to_str()` the stack would look like Figure below:

Each time we make a call to `to_str()`, we push a character on the stack. Returning to the previous example we can see that after the fourth call to `to_str()` the stack would look like Figure below:



Each time we make a call to `to_str()`, we push a character on the stack. Returning to the previous example we can see that after the fourth call to `to_str()` the stack would look like Figure below:



Notice that now we can simply pop the characters off the stack and concatenate them into the final result, `"1010"`.

The previous example gives us some insight into how `C++` implements a recursive function call. When a function is called in `Python`, a stack frame is allocated to handle the local variables of the function.

The previous example gives us some insight into how `C++` implements a recursive function call. When a function is called in `Python`, a stack frame is allocated to handle the local variables of the function.

When the function returns, the return value is left on top of the stack for the calling function to access.

Figure below illustrates the call stack after the return statement:

Figure below illustrates the call stack after the return statement:

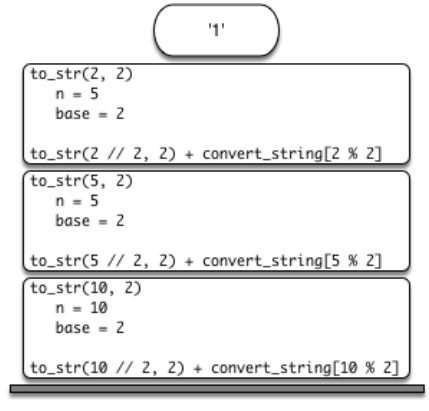
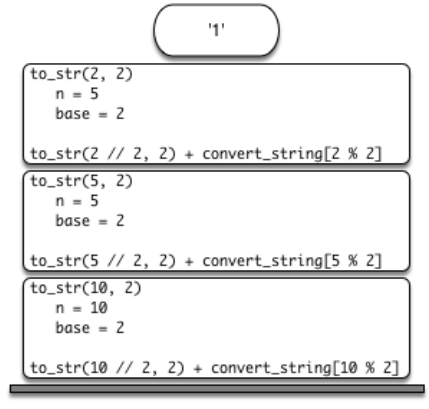


Figure below illustrates the call stack after the return statement:



Notice that the call to `to_str(2 // 2, 2)` leaves a return value of `"1"` on the stack. This return value is then used in place of the function call (`to_str(1, 2)`) in the expression `"1" + convert_string[2 % 2]` , which will leave the string `"10"` on the top of the stack.

In this way, the `C++` call stack takes the place of the stack we used explicitly. In our list summing example, you can think of the return value on the stack taking the place of an accumulator variable.

In this way, the `C++` call stack takes the place of the stack we used explicitly. In our list summing example, you can think of the return value on the stack taking the place of an accumulator variable.

The stack frames also provide a scope for the variables used by the function. Even though we are calling the same function over and over, each call creates a new scope for the variables that are local to the function.

5.10 Tower of Hanoi

The Tower of Hanoi puzzle was invented by the French mathematician Edouard Lucas in 1883. He was inspired by a legend that tells of a Hindu temple where the puzzle was presented to young priests.

The Tower of Hanoi puzzle was invented by the French mathematician Edouard Lucas in 1883. He was inspired by a legend that tells of a Hindu temple where the puzzle was presented to young priests.

At the beginning of time, the priests were given three poles and a stack of 64 gold disks, each disk a little smaller than the one beneath it. Their assignment was to transfer all 64 disks from one of the three poles to another, with two important constraints.

The Tower of Hanoi puzzle was invented by the French mathematician Edouard Lucas in 1883. He was inspired by a legend that tells of a Hindu temple where the puzzle was presented to young priests.

At the beginning of time, the priests were given three poles and a stack of 64 gold disks, each disk a little smaller than the one beneath it. Their assignment was to transfer all 64 disks from one of the three poles to another, with two important constraints.

1. They could only move one disk at a time.
2. They could never place a larger disk on top of a smaller one.

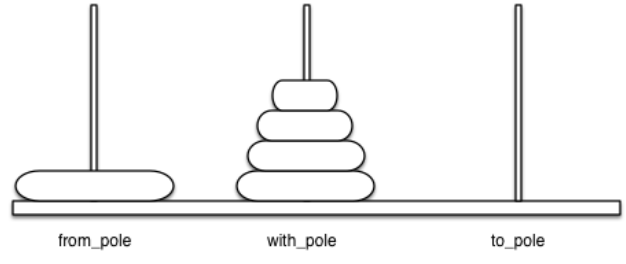
The priests worked very efficiently, day and night, moving one disk every second. When they finished their work, the legend said, the temple would crumble into dust and the world would vanish.

The priests worked very efficiently, day and night, moving one disk every second. When they finished their work, the legend said, the temple would crumble into dust and the world would vanish.

Although the legend is interesting, you need not worry about the world ending any time soon. The number of moves required to correctly move a tower of 64 disks is $2^{64} - 1 = 18,446,744,073,709,551,615$. At a rate of one move per second, that is $584,942,417,355$ years!

Figure below shows an example of a configuration of disks in the middle of a move from the first peg to the third. Notice that, as the rules specify, the disks on each peg are stacked so that smaller disks are always on top of the larger disks:

Figure below shows an example of a configuration of disks in the middle of a move from the first peg to the third. Notice that, as the rules specify, the disks on each peg are stacked so that smaller disks are always on top of the larger disks:



Let's think about this problem from the bottom up. Suppose you have a tower of five disks, originally on peg one. If you already knew how to move a tower of four disks to peg two, you could then easily move the bottom disk to peg three, and then move the tower of four from peg two to peg three.

Let's think about this problem from the bottom up. Suppose you have a tower of five disks, originally on peg one. If you already knew how to move a tower of four disks to peg two, you could then easily move the bottom disk to peg three, and then move the tower of four from peg two to peg three.

But what if you do not know how to move a tower of height four? Suppose that you knew how to move a tower of height three to peg three; then it would be easy to move the fourth disk to peg two and move the three from peg three on top of it.

Let's think about this problem from the bottom up. Suppose you have a tower of five disks, originally on peg one. If you already knew how to move a tower of four disks to peg two, you could then easily move the bottom disk to peg three, and then move the tower of four from peg two to peg three.

But what if you do not know how to move a tower of height four? Suppose that you knew how to move a tower of height three to peg three; then it would be easy to move the fourth disk to peg two and move the three from peg three on top of it.

But what if you still do not know how to do this? Surely you would agree that moving a single disk to peg three is easy enough, trivial you might even say. This sounds like a base case in the making.

Here is a high-level outline of how to move a tower of height from the starting pole to the goal pole, using an intermediate pole:

Here is a high-level outline of how to move a tower of height h from the starting pole to the goal pole, using an intermediate pole:

1. Move a tower of height $h - 1$ from the starting pole to an intermediate pole via the goal pole.

Here is a high-level outline of how to move a tower of height h from the starting pole to the goal pole, using an intermediate pole:

1. Move a tower of height $h - 1$ from the starting pole to an intermediate pole via the goal pole.

2. Move the remaining disk from the starting pole to the final pole.

Here is a high-level outline of how to move a tower of height h from the starting pole to the goal pole, using an intermediate pole:

1. Move a tower of height $h - 1$ from the starting pole to an intermediate pole via the goal pole.

2. Move the remaining disk from the starting pole to the final pole.

3. Move the tower of height $h - 1$ from the intermediate pole to the goal pole via the starting pole.

Here is a high-level outline of how to move a tower of height h from the starting pole to the goal pole, using an intermediate pole:

1. Move a tower of height $h - 1$ from the starting pole to an intermediate pole via the goal pole.

2. Move the remaining disk from the starting pole to the final pole.

3. Move the tower of height $h - 1$ from the intermediate pole to the goal pole via the starting pole.

The simplest Tower of Hanoi problem is a tower of one disk. In this case, we need move only a single disk to its final destination. A tower of one disk will be our base case. In addition, the steps outlined above move us toward the base case by reducing the height of the tower in steps 1 and 3.

In [7]:

```cpp
#include <iostream>
#include <string>
using namespace std;

void moveDisk(string fromP, string toP) {
    cout << "moving disk from " << fromP << " to " << toP << endl;
}
void moveTower(int height, string fromPole, string toPole, string withPole) {
    if (height >= 1) {
        moveTower(height - 1, fromPole, withPole, toPole);
        moveDisk(fromPole, toPole);
        moveTower(height - 1, withPole, toPole, fromPole);
    }
}

int main() { moveTower(3, "A", "B", "C"); }
```

moving disk from A to B

moving disk from A to C

moving disk from B to C

moving disk from A to B

moving disk from C to A

moving disk from C to B

moving disk from A to B

In [7]:

```cpp
#include <iostream>
#include <string>
using namespace std;

void moveDisk(string fromP, string toP) {
    cout << "moving disk from " << fromP << " to " << toP << endl;
}
void moveTower(int height, string fromPole, string toPole, string withPole) {
    if (height >= 1) {
        moveTower(height - 1, fromPole, withPole, toPole);
        moveDisk(fromPole, toPole);
        moveTower(height - 1, withPole, toPole, fromPole);
    }
}

int main() { moveTower(3, "A", "B", "C"); }
```

moving disk from A to B

moving disk from A to C

moving disk from B to C

moving disk from A to B

moving disk from C to A

moving disk from C to B

moving disk from A to B

The base case is the tower of height 0; in this case there is nothing to do, so the `move_tower()` function returns. The important thing to remember about handling the base case this way is that simply returning from `move_tower()` is what finally allows the `move_disk()` function to be called.

Now that you have seen the code for both `move_tower()` and `move_disk()`, you may be wondering why we do not have a data structure that explicitly keeps track of what disks are on what poles.

Now that you have seen the code for both `move_tower()` and `move_disk()`, you may be wondering why we do not have a data structure that explicitly keeps track of what disks are on what poles.

Here is a hint: if you were going to explicitly keep track of the disks, you would probably use three `Stack` objects, one for each pole. The answer is that `Python` provides the stacks that we need implicitly through the call stack!

5.11 Exploring a Maze

In this section we will look at a problem that has relevance to the expanding world of robotics: how do you find your way out of a maze?

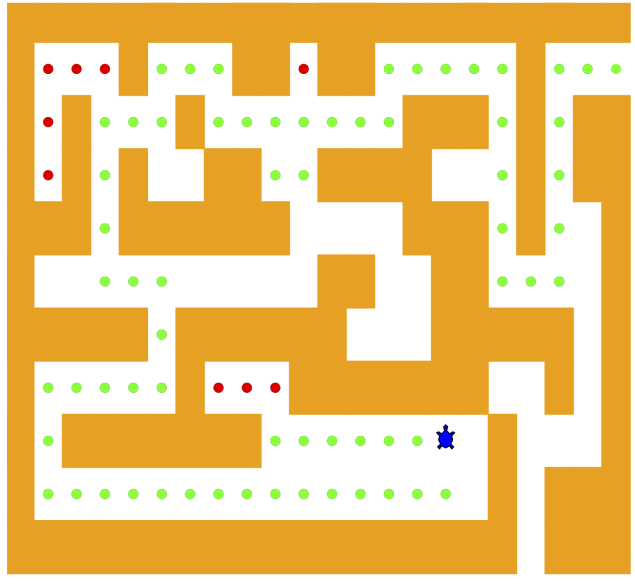
In this section we will look at a problem that has relevance to the expanding world of robotics: how do you find your way out of a maze?

The problem we want to solve is to help our turtle find its way out of a virtual maze. The maze problem has roots as deep as the Greek myth about Theseus, who was sent into a maze to kill the Minotaur.

In this section we will look at a problem that has relevance to the expanding world of robotics: how do you find your way out of a maze?

The problem we want to solve is to help our turtle find its way out of a virtual maze. The maze problem has roots as deep as the Greek myth about Theseus, who was sent into a maze to kill the Minotaur.

Theseus used a ball of thread to help him find his way back out again once he had finished off the beast. In our problem we will assume that our turtle is dropped down somewhere into the middle of the maze and must find its way out.



To make it easier for us we will assume that our maze is divided up into squares. Each square of the maze is either open or occupied by a section of wall. The turtle can only pass through the open squares of the maze. If the turtle bumps into a wall, it must try a different direction. Here is the procedure:

To make it easier for us we will assume that our maze is divided up into squares. Each square of the maze is either open or occupied by a section of wall. The turtle can only pass through the open squares of the maze. If the turtle bumps into a wall, it must try a different direction. Here is the procedure:

1. From our starting position we will first try going north one square and then **recursively try our procedure from there**.

To make it easier for us we will assume that our maze is divided up into squares. Each square of the maze is either open or occupied by a section of wall. The turtle can only pass through the open squares of the maze. If the turtle bumps into a wall, it must try a different direction. Here is the procedure:

1. From our starting position we will first try going north one square and then **recursively try our procedure from there**.

2. If we are not successful by trying a northern path as the first step then we will take a step to the south and recursively repeat our procedure.

3. If south does not work then we will try a step to the west as our first step and recursively apply our procedure.

3. If south does not work then we will try a step to the west as our first step and recursively apply our procedure.

4. If north, south, and west have not been successful then we will apply the procedure recursively from a position one step to our east.

3. If south does not work then we will try a step to the west as our first step and recursively apply our procedure.

4. If north, south, and west have not been successful then we will apply the procedure recursively from a position one step to our east.

5. If none of these directions works then there is no way to get out of the maze and we fail.

Now that sounds easy, but there are a couple of details to talk about! Suppose we take our first recursive step by going north. By following our procedure, our next step would also be to the north. But if the north is blocked by a wall, we must look at the next step of the procedure and try going to the south.

Now that sounds easy, but there are a couple of details to talk about! Suppose we take our first recursive step by going north. By following our procedure, our next step would also be to the north. But if the north is blocked by a wall, we must look at the next step of the procedure and try going to the south.

Unfortunately, that step to the south brings us right back to our original starting place. If we apply the recursive procedure from there, we will just go back one step to the North and be in an infinite loop! So we must have a strategy to remember where we have been!

Now that sounds easy, but there are a couple of details to talk about! Suppose we take our first recursive step by going north. By following our procedure, our next step would also be to the north. But if the north is blocked by a wall, we must look at the next step of the procedure and try going to the south.

Unfortunately, that step to the south brings us right back to our original starting place. If we apply the recursive procedure from there, we will just go back one step to the North and be in an infinite loop! So we must have a strategy to remember where we have been!

In this case we will assume that we have a **bag of bread crumbs we can drop along our way.** If we take a step in a certain direction and find that there is a bread crumb already on that square, we know that we should immediately back up and try the next direction in our procedure. As we will see when we look at the code for this algorithm, **backing up is as simple as returning from a recursive function call.**

As we do for all recursive algorithms, let us review the base cases. Some of them you may already have guessed based on the description in the previous paragraph. In this algorithm, there are four base cases to consider:

As we do for all recursive algorithms, let us review the base cases. Some of them you may already have guessed based on the description in the previous paragraph. In this algorithm, there are four base cases to consider:

1. The turtle has run into a wall. Since the square is occupied by a wall, no further exploration can take place.

2. The turtle has found a square that has already been explored. We do not want to continue exploring from this position so we don't get into a loop.

As we do for all recursive algorithms, let us review the base cases. Some of them you may already have guessed based on the description in the previous paragraph. In this algorithm, there are four base cases to consider:

1. The turtle has run into a wall. Since the square is occupied by a wall, no further exploration can take place.

2. The turtle has found a square that has already been explored. We do not want to continue exploring from this position so we don't get into a loop.

3. We have found an outside edge, not occupied by a wall. In other words, we have found an exit from the maze.

4. We have explored a square unsuccessfully in all four directions.

Now we will considering the following maze:

```
++++++++++++++++++++++
+   +   ++ ++     +
+ +   +       +++ + ++
+ + +  ++  ++++   + ++
+++ ++++++    +++ +  +
+          ++  ++    +
+++++ ++++++   +++++ +
+     +   +++++++  + +
+ +++++++      S +   +
+                + +++
++++++++++++++++++ +++
```

The `Maze` object will provide the following methods for us to use in writing our search algorithm:

1. `__init__` Reads in a data file representing a maze, initializes the internal representation of the maze, and finds the starting position for the turtle.

2. `draw_maze()` Draws the maze in a window on the screen.

The `Maze` object will provide the following methods for us to use in writing our search algorithm:

1. `__init__` Reads in a data file representing a maze, initializes the internal representation of the maze, and finds the starting position for the turtle.

2. `draw_maze()` Draws the maze in a window on the screen.

3. `update_position()` Updates the internal representation of the maze and changes the position of the turtle in the window.

4. `is_exit()` Checks to see if the current position is an exit from the maze.

The `Maze` object will provide the following methods for us to use in writing our search algorithm:

1. `__init__` Reads in a data file representing a maze, initializes the internal representation of the maze, and finds the starting position for the turtle.

2. `draw_maze()` Draws the maze in a window on the screen.

3. `update_position()` Updates the internal representation of the maze and changes the position of the turtle in the window.

4. `is_exit()` Checks to see if the current position is an exit from the maze.

The `Maze` class also overloads the index operator `[]` so that our algorithm can easily access the status of any particular square.

We represent the maze with a few named constants — the `C++` version prints its progress as text instead of animating with turtle graphics:

```
const char START = 'S';
const char OBSTACLE = '+';
const char TRIED = '.';
const char DEAD_END = '-';
const char PART_OF_PATH = 'O';
```

We represent the maze with a few named constants — the `C++` version prints its progress as text instead of animating with turtle graphics:

```
const char START = 'S';
const char OBSTACLE = '+';
const char TRIED = '.';
const char DEAD_END = '-';
const char PART_OF_PATH = 'O';
```

The `__init__` method takes the name of a file as its only parameter. This file is a text file that represents a maze by using "+" characters for walls, spaces for open squares, and the letter "S" to indicate the starting position.

```
class Maze {
    public:
        Maze(string mazeFilename) {
            ifstream mazeFile(mazeFilename);
            string line;
            while (getline(mazeFile, line)) {
                mazeList.push_back(line);
            }
            rowsInMaze = mazeList.size();
            columnsInMaze = mazeList[0].size();
            ...
```

```
class Maze {
    public:
        Maze(string mazeFilename) {
            ifstream mazeFile(mazeFilename);
            string line;
            while (getline(mazeFile, line)) {
                mazeList.push_back(line);
            }
            rowsInMaze = mazeList.size();
            columnsInMaze = mazeList[0].size();
            ...

            ...
            for (int row = 0; row < rowsInMaze; row++) {
                size_t col = mazeList[row].find(START);
                if (col != string::npos) {
                    startRow = row;
                    startCol = col;
                    break;
                }
            }
        }

        int startRow;
        int startCol;
    private:
        vector<string> mazeList;
        int rowsInMaze;
        int columnsInMaze;
};
```

The Python version draws the maze with turtle graphics; here we simply print the character grid — each update is visible in the printed maze at the end:

```
void updatePosition(int row, int col, char val) {
    mazeList[row][col] = val;
}

void print() {
    for (string& row : mazeList) {
        cout << row << endl;
    }
}
```

```
bool isExit(int row, int col) {
    return (row == 0 || row == rowsInMaze - 1 ||
            col == 0 || col == columnsInMaze - 1);
}

char get(int row, int col) {
    return mazeList[row][col];
}
```

```
bool searchFrom(Maze& maze, int row, int column) {
    // Base Case return values:
    //  1. We have run into an obstacle, return false
    if (maze.get(row, column) == OBSTACLE) {
        return false;
    }
    //  2. We have found an already explored square
    if (maze.get(row, column) == TRIED || maze.get(row, column) == DEAD_END)
{
        return false;
    }
    //  3. We have found an exit
    if (maze.isExit(row, column)) {
        maze.updatePosition(row, column, PART_OF_PATH);
        return true;
    }
    maze.updatePosition(row, column, TRIED);
    ...
```

```
bool searchFrom(Maze& maze, int row, int column) {
    // Base Case return values:
    //  1. We have run into an obstacle, return false
    if (maze.get(row, column) == OBSTACLE) {
        return false;
    }
    //  2. We have found an already explored square
    if (maze.get(row, column) == TRIED || maze.get(row, column) == DEAD_END)
{
        return false;
    }
    //  3. We have found an exit
    if (maze.isExit(row, column)) {
        maze.updatePosition(row, column, PART_OF_PATH);
        return true;
    }
    maze.updatePosition(row, column, TRIED);
    ...
```

```
    ...
    // Otherwise, use logical short circuiting to try each direction
    bool found = searchFrom(maze, row - 1, column)
              || searchFrom(maze, row + 1, column)
              || searchFrom(maze, row, column - 1)
              || searchFrom(maze, row, column + 1);
    if (found) {
        maze.updatePosition(row, column, PART_OF_PATH);
    } else {
        maze.updatePosition(row, column, DEAD_END);
    }
```

```
    return found;
}
```

(complete listing: `dscpp/maze.hpp`)

You will notice that in the recursive step there are four recursive calls to `searchFrom()`. It is hard to predict how many of these will be used since they are connected by `||` — if the first call succeeds, short-circuit evaluation skips the rest.

You will notice that in the recursive step there are four recursive calls to `searchFrom()`. It is hard to predict how many of these will be used since they are connected by `||` — if the first call succeeds, short-circuit evaluation skips the rest.

If the first call to `search_from()` returns `True` then none of the last three calls would be needed. You can interpret this as meaning that a step to `(row - 1, column)` (or north if you want to think geographically) is on the path leading out of the maze.

You will notice that in the recursive step there are four recursive calls to `searchFrom()`. It is hard to predict how many of these will be used since they are connected by `||` — if the first call succeeds, short-circuit evaluation skips the rest.

If the first call to `search_from()` returns `True` then none of the last three calls would be needed. You can interpret this as meaning that a step to `(row - 1, column)` (or north if you want to think geographically) is on the path leading out of the maze.

If there is not a good path leading out of the maze to the north then the next recursive call is tried, this one to the south. If south fails then try west, and finally east. If all four recursive calls return `False` then we have found a dead end.

In [8]:

```cpp
#include <iostream>
#include "dscpp/maze.hpp"   // Maze class + searchFrom (md listing above)
using namespace std;

int main() {
    Maze myMaze("maze2.txt");
    searchFrom(myMaze, myMaze.startRow, myMaze.startCol);
    myMaze.print();   // O = path, . = tried, - = dead end
    return 0;
}
```

++++++++++++++++++++++

+-OO+OOO++ ++ +

OOOOOO+OOO ++++++++++

+-+-OO ++O ++++ +++ ++

+-+-OO+ +O++ OO +++ +

+---OO OO++OO++ + +

+++++-+ +-OOOOO++ + +

+++++-+++--+-+OO++ +

+----------+-+ O+ + +

+++++-+--+-+-+ + +

++++++++++++++++++++++

In [8]:

```cpp
#include <iostream>
#include "dscpp/maze.hpp"   // Maze class + searchFrom (md listing above)
using namespace std;

int main() {
    Maze myMaze("maze2.txt");
    searchFrom(myMaze, myMaze.startRow, myMaze.startCol);
    myMaze.print();   // O = path, . = tried, - = dead end
    return 0;
}
```

```
++++++++++++++++++++++
+-OO+OOO++ ++ +
OOOOOO+OOO ++++++++++
+-+-OO ++O ++++ +++ ++
+-+-OO+ +O++ OO +++ +
+---OO OO++OO++ + +
++++++-+ +-OOOOO++ + +
++++++-+++--+-+OO++ +
+----------+-+ O+ + +
+++++-+--+-+-+ + +
++++++++++++++++++++++
```

The maze file `maze2.txt` (already in the course folder):

```
++++++++++++++++++++++
+   +   ++ ++        +
      +     ++++++++++
+ +    ++  ++++ +++ ++
+ +   + + ++    +++  +
+          ++  ++  + +
+++++ + +      ++  + +
+++++ +++  + +  ++   +
```

```
+          + + S+ +  +
+++++ +  + + +     + +
++++++++++++++++++++++
```

In the printed result, `O` marks the successful path, `.` are tried squares, and `-` are dead ends. For the animated turtle version, run the original Python demo: `python maze.py`.

In [ ]:

```
from jupytercards import display_flashcards
fpath = "flashcards/"
display_flashcards(fpath + 'ch6.json')
```

References

1. Textbook: *Problem Solving with Algorithms and Data Structures using C++* (cppds), Chapter 5 (Recursion) — https://runestone.academy/ns/books/published/cppds/index.html